

Zadanie 1: Dziedziczenie – rodzina zwierząt

Cel dydaktyczny: Zrozumienie, jak jedna klasa może dziedziczyć cechy od innej (wykorzystanie `super()`).

Treść zadania:

1. Stwórz klasę bazową `Zwierze`, która w `__init__` przyjmuje atrybut `imie`. Dodaj do niej metodę `daj_glos()`, która wypisze "Nieznany dźwięk".
2. Stwórz klasę `Pies`, która dziedziczy po klasie `Zwierze`.
3. W klasie `Pies` stwórz własny konstruktor `__init__`, który przyjmuje `imie` oraz `rasa`. Użyj funkcji `super()`, aby przekazać `imie` do klasy bazowej.
4. Nadpisz (zdefiniuj ponownie) metodę `daj_glos()` w klasie `Pies`, aby wypisywała: *"Hau! Hau! Jestem [imie] i moja rasa to [rasa]"*.
5. Przetestuj działanie, tworząc obiekt klasy `Pies` i wywołując metodę `daj_glos()`.

Zadanie 2: Hermetyzacja – tajne hasło

Cel dydaktyczny: Ukrywanie danych wewnątrz obiektu za pomocą atrybutów prywatnych i tworzenie tzw. getterów/setterów.

Treść zadania:

1. Stwórz klasę `Sejff`. W konstruktorze ustaw atrybut prywatny `__zawartosc` (np. przypisz mu wartość "100 sztuk złota") oraz atrybut prywatny `__pin` (np. "1234").
2. Zauważ, że wpisanie `print(moj_sejff.__zawartosc)` wyrzuci błąd.
3. Napisz publiczną metodę `otworz(podany_pin)`. Metoda ta ma sprawdzić, czy `podany_pin` zgadza się z `__pin`.
4. Jeśli PIN jest poprawny, metoda ma zwrócić (lub wypisać) `__zawartosc`. Jeśli błędny – wypisać "Dostęp odmówiony!".

Zadanie 3: Metody magiczne – ładne wyświetlanie (`__str__`)

Cel dydaktyczny: Nadpisywanie metod wbudowanych Pythona, aby obiekty "współpracowały" z funkcją `print()`.

Treść zadania:

1. Stwórz klasę `Uczen`. Konstruktor ma przyjmować `imie` i `nazwisko` oraz tworzyć pustą listę `oceny`.
2. Dodaj metodę `dodaj_ocene(ocena)`.
3. Dodaj metodę `policz_srednia()`, która zwraca średnią ocen (pamiętaj o zabezpieczeniu przed dzieleniem przez 0, jeśli lista jest pusta).
4. Zdefiniuj w klasie metodę `__str__(self)`. Ma ona zwracać ładnie sformatowany tekst (string), np.: *"Uczeń: [Imię] [Nazwisko], Średnia ocen: [Średnia]"*.
5. Stwórz ucznia, dodaj mu trzy oceny i wypisz obiekt zwykłym `print(uczen)`.

Zadanie 4: Kompozycja – obiekty w obiektach

Cel dydaktyczny: Współpraca pomiędzy wieloma różnymi klasami (przechowywanie obiektów jednej klasy w liście innej klasy).

Treść zadania:

1. Skopiuj klasę `Ksiazka` z Zadania 1 z poprzedniej lekcji.
2. Stwórz nową klasę `Biblioteka`, która w `__init__` posiada tylko jeden atrybut: pustą listę `katalog_ksiazek`.
3. W klasie `Biblioteka` dodaj metodę `dodaj_do_katalogu(ksiazka)`, która przyjmuje **obiekt** klasy `Ksiazka` i dodaje go do listy.
4. Dodaj metodę `wyswietl_katalog()`, która przejdzie pętlą `for` po liście książek i wypisze ich tytuły oraz autorów.
5. Stwórz dwa obiekty `Ksiazka`, stwórz obiekt `Biblioteka`, dodaj książki do biblioteki i wyświetl katalog.

Zadanie 5: Atrybuty klasy vs Atrybuty instancji

Cel dydaktyczny: Zrozumienie zmiennych współdzielonych przez wszystkie obiekty danej klasy.

Treść zadania:

1. Stwórz klasę `Pracownik`.
2. Bezpośrednio w ciele klasy (przed `__init__`) zdefiniuj atrybut klasy `liczba_pracownikow = 0`.
3. W konstruktorze `__init__` przypisz pracownikowi `imie` oraz `stanowisko`. Dodatkowo, przy każdym wywołaniu konstruktora, zwiększ wartość `Pracownik.liczba_pracownikow` o 1.
4. Stwórz trzech różnych pracowników.
5. Na koniec wypisz `Pracownik.liczba_pracownikow`, aby udowodnić, że program sam policzył stworzone obiekty i wynik wynosi 3.

Zadanie 6: Sklep i walidacja danych (Zgłaszanie błędów/Wyjątki)

Cel dydaktyczny: Sprawdzanie poprawności danych wewnątrz metod klasy i rzucanie wyjątków (błędów) lub wyświetlanie ostrzeżeń.

Treść zadania:

1. Stwórz klasę `Produkt` z atrybutami `nazwa`, `cena` oraz `ilosc_w_magazynie`.
2. Zabezpiecz konstruktor: jeśli przy tworzeniu obiektu podana `cena` lub `ilosc_w_magazynie` jest mniejsza od 0, użyj instrukcji `raise ValueError("Cena i ilość nie mogą być ujemne!")`.
3. Dodaj metodę `kup(ilosc)`. Metoda ma sprawdzić czy chcemy kupić poprawną ilość (większą od zera) i czy w ogóle mamy tyle sztuk w magazynie.
4. Jeśli tak, zmniejsz `ilosc_w_magazynie` i wypisz całkowity koszt (`ilosc * cena`). Jeśli nie, wypisz komunikat "Niewystarczająca ilość w magazynie!".
5. Przetestuj program. Spróbuj stworzyć produkt z ujemną ceną (aby zobaczyć błąd), a następnie stwórz poprawny produkt i przetestuj udane oraz nieudane zakupy.